GoFixCarz
Customer-Only API Requirements

DRAFT FOR CLIENT APPROVAL

Version 1.1 | Customer-only scope | Prepared 7 October 2026

Request payloads, sample responses, business rules and approval decisions

Source: GoFixCarz Customer App UI and User Stories workbook - 38 user stories across 15 epics and 21 UI screens.

This revision includes customer authentication and customer-owned operations only. Public discovery/content URLs, garage-only operations and provider callback URLs are intentionally excluded. It supersedes the broader approval draft for this review.

This document proposes the customer-app API contract. It does not certify existing implementation or authorize development, data changes, payment configuration or publishing.

All example people, contacts, identifiers, amounts and records are synthetic. Credential fields contain placeholders only.

Prepared for client review. Please approve the scope and resolve the decision register before implementation.

Contents

1. Scope and conventions

2. Endpoint index

3. Detailed API contracts

4. Customer-only scope boundary

5. Story and screen traceability

6. Approval decision register

7. Client sign-off

The detailed sections follow the index order. Word headings provide navigation; PDF bookmarks provide section navigation.

1. Scope and conventions

Proposed base path: /api/v1/customer-app. Endpoint paths in the detailed sections are relative to this base unless explicitly stated otherwise.

Access: Customer = Authorization: Bearer <access_token>; Customer pre-login = registration/login/OTP/recovery operations that cannot require an existing session; Refresh/Recovery credential = the purpose-specific proof shown in the request. Every private record is scoped to the authenticated customer. No public browsing or content URL is included.

GET requests have no body. Path placeholders are required opaque IDs (UUIDs are recommended in implementation); example short IDs improve readability. Reject invalid IDs and unknown fields. Do not accept client ownership, role, status or monetary authority fields.

Default pagination: page=1, page_size=20; proposed bounds page >= 1 and page_size 1..100. Cursor-based message endpoints use opaque next_cursor. Lists show items and total; [] is a valid empty result.

Times are ISO 8601 with UTC Z or explicit offsets. Garage appointment rules use its IANA timezone. Monetary amounts are integer paise; currency is INR in these examples. Monetary precision and tax calculations are server-controlled.

POST creation normally returns 201; delivery requests return 202; reads/updates/actions return 200. Binary downloads do not use the JSON envelope. Provider acceptance is not confirmation of OTP delivery or payment settlement.

Shared JSON success envelope (all detailed JSON examples use this complete structure):

```
{
  "success": true,
  "message": "Booking retrieved",
  "data": {
    "id": "booking_001",
    "status": "PENDING"
  }
}
```

Shared JSON error envelope:

```
{
  "success": false,
  "message": "The selected slot is no longer available",
  "error": {
    "code": "SLOT_UNAVAILABLE",
    "details": {
      "slot_id": "slot_001"
    }
  }
}
```

Every endpoint-specific error code below uses this envelope, with a clear safe message and relevant field details. Common errors: 401 AUTH_REQUIRED/SESSION_EXPIRED; 403 FORBIDDEN; 404 RESOURCE_NOT_FOUND; 422 VALIDATION_ERROR; 429 RATE_LIMITED. Return Retry-After for rate limits. Another customer's private record should appear as 404 to avoid existence leaks.

Examples are snapshots across a journey, not simultaneous records: booking_001 progresses from PENDING to ACCEPTED to CONVERTED; job_001 progresses to COMPLETED; invoice_001 moves from UNPAID to PAID after payment_002 succeeds. booking_002 illustrates an independent cancelled booking.

Priority: source stories remain Must Have or Should Have as recorded in the traceability appendix. Excluded public requirements are explicitly identified rather than claimed as covered. Password operations are conditional.

2. Endpoint index

ID	Endpoint (relative to base path)	Access
API-01	POST /auth/register	Customer pre-login
API-02	POST /auth/otp/send	Customer pre-login
API-03	POST /auth/otp/resend	Customer pre-login
API-04	POST /auth/otp/verify	Customer pre-login
API-05	POST /auth/refresh	Refresh credential
API-06	GET /auth/session	Customer
API-07	POST /auth/logout	Customer
API-08	POST /auth/password/login	Customer pre-login
API-09	PUT /auth/password	Customer
API-10	POST /auth/password/reset/request	Customer pre-login
API-11	POST /auth/password/reset/confirm	Recovery credential
API-12	GET /me	Customer
API-13	PATCH /me	Customer
API-14	POST /me/contact-change	Customer
API-15	POST /me/contact-change/verify	Customer
API-16	PUT /me/photo	Customer
API-17	DELETE /me/photo	Customer
API-18	POST /vehicles	Customer
API-19	GET /vehicles	Customer
API-20	GET /vehicles/{vehicle_id}	Customer
API-21	PATCH /vehicles/{vehicle_id}	Customer
API-22	DELETE /vehicles/{vehicle_id}	Customer
API-23	PUT /vehicles/{vehicle_id}/default	Customer
API-24	GET /vehicles/{vehicle_id}/service-history	Customer
API-25	POST /attachments	Customer
API-26	GET /attachments/{attachment_id}/download	Customer
API-27	DELETE /attachments/{attachment_id}	Customer
API-28	GET /garages/{garage_id}/availability	Customer
API-29	POST /bookings/quote	Customer
API-30	POST /bookings	Customer
API-31	GET /bookings	Customer
API-32	GET /bookings/{booking_id}	Customer
API-33	GET /bookings/{booking_id}/timeline	Customer
API-34	POST /bookings/{booking_id}/cancel	Customer
API-35	GET /jobs	Customer
API-36	GET /jobs/{job_id}	Customer
API-37	GET /jobs/{job_id}/timeline	Customer
API-38	GET /invoices	Customer
API-39	GET /invoices/{invoice_id}	Customer
API-40	GET /invoices/{invoice_id}/pdf	Customer
API-41	POST /invoices/{invoice_id}/payment-attempts	Customer
API-42	GET /payments/{payment_id}	Customer

ID	Endpoint (relative to base path)	Access
API-43	GET /invoices/{invoice_id}/payment-attempts	Customer
API-44	GET /payments/{payment_id}/receipt	Customer
API-45	PUT /devices/{installation_id}	Customer
API-46	DELETE /devices/{installation_id}	Customer
API-47	GET /notifications	Customer
API-48	PATCH /notifications/{notification_id}	Customer
API-49	POST /notifications/read-all	Customer
API-50	POST /conversations	Customer
API-51	GET /conversations	Customer
API-52	GET /conversations/{conversation_id}/messages	Customer
API-53	POST /conversations/{conversation_id}/messages	Customer
API-54	POST /conversations/{conversation_id}/read	Customer
API-55	POST /jobs/{job_id}/review	Customer
API-56	GET /me/reviews	Customer
API-57	PATCH /reviews/{review_id}	Customer
API-58	POST /support/tickets	Customer
API-59	GET /support/tickets	Customer
API-60	GET /support/tickets/{ticket_id}	Customer
API-61	GET /support/tickets/{ticket_id}/messages	Customer
API-62	POST /support/tickets/{ticket_id}/messages	Customer

3. Detailed API contracts

Authentication and sessions

Traceability: US-1, US-2, US-3, US-4, US-5 | UI-01, UI-02, UI-03, UI-21

• Core registration/login is OTP-based. Account creation occurs only after verification. Registration supports mobile OR email in this proposal; the UI discrepancy needs approval.

• Mobile identifiers use E.164; email identifiers are normalized. Terms consent and the published terms version are required for registration.

• OTP send/resend limits and failed-attempt limits must protect both channels and combine identifier, IP and device signals. Proposed OTP expiry: 5 minutes; resend cooldown: 30 seconds; maximum verification attempts: 5 per challenge. Delivery submission is not proof of delivery.

• Challenge IDs bind identifier, purpose and pending registration. Resending invalidates the prior challenge. OTPs are single-use. Successful login responses are issued only after validation.

• Access-token lifetime of 15 minutes is a proposed default, not an approved setting. Refresh-token lifetime remains a decision. Rotate refresh tokens, detect reuse, revoke the token family/session, and store mobile credentials in secure device storage.

• Logout revokes the current session and refresh credentials. Protected requests must honor session revocation; clearing client storage alone is insufficient.

API-01 | Start verified registration

POST /api/v1/customer-app/auth/register

Access: Customer pre-login | Success HTTP: 202

Fields and rules: All illustrated fields required. Name must be nonblank; channel is mobile or email. No customer account is created yet.

Request payload:

```
{
  "name": "Demo Customer",
  "channel": "mobile",
  "identifier": "+919000000001",
  "terms_version": "2026-10",
  "terms_accepted": true
}
```

Sample success response:

```
{
  "success": true,
  "message": "Start verified registration successful",
  "data": {
    "challenge_id": "challenge_register_001",
    "purpose": "REGISTER",
    "expires_in_seconds": 300,
    "resend_after_seconds": 30
  }
}
```

Representative failures: 409 ACCOUNT_ALREADY_EXISTS; 422 VALIDATION_ERROR; 429 RATE_LIMITED. Common authentication/validation rules also apply where relevant.

API-02 | Request a login OTP

POST /api/v1/customer-app/auth/otp/send

Access: Customer pre-login | Success HTTP: 202

Fields and rules: All fields required. Purpose is LOGIN; registration and contact changes use their dedicated flows. Use generic acceptance behavior to reduce account enumeration.

Request payload:

```
{
  "channel": "mobile",
  "identifier": "+919000000001",
  "purpose": "LOGIN"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Request a login OTP successful",
  "data": {
    "challenge_id": "challenge_login_001",
    "expires_in_seconds": 300,
    "resend_after_seconds": 30
  }
}
```

Representative failures: 422 VALIDATION_ERROR; 429 RATE_LIMITED. Common authentication/validation rules also apply where relevant.

API-03 | Resend OTP

POST /api/v1/customer-app/auth/otp/resend

Access: Customer pre-login | Success HTTP: 202

Fields and rules: challenge_id required. Enforce cooldown and cumulative limits; previous challenge becomes unusable.

Request payload:

```
{
  "challenge_id": "challenge_login_001"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Resend OTP successful",
  "data": {
    "challenge_id": "challenge_login_002",
    "expires_in_seconds": 300,
    "resend_after_seconds": 30
  }
}
```

Representative failures: 400 CHALLENGE_INVALID; 429 RATE_LIMITED. Common authentication/validation rules also apply where relevant.

API-04 | Verify OTP and establish session

POST /api/v1/customer-app/auth/otp/verify

Access: Customer pre-login | Success HTTP: 200

Fields and rules: challenge_id and otp required; device_name optional. OTP is a string, preserving leading zeroes. Registration challenge verification creates the account and session.

Request payload:

```
{
  "challenge_id": "challenge_login_002",
  "otp": "123456",
  "device_name": "Customer Phone"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Verify OTP and establish session successful",
  "data": {
    "customer": {
      "id": "customer_001",
      "name": "Demo Customer"
    },
    "access_token": "<access_token>",
    "refresh_token": "<refresh_token>",
    "token_type": "Bearer",
    "expires_in_seconds": 900,
    "session_id": "session_001"
  }
}
```

Representative failures: 400 OTP_INVALID; 400 OTP_EXPIRED; 429 OTP_ATTEMPTS_EXCEEDED. Common authentication/validation rules also apply where relevant.

API-05 | Rotate refresh credentials

POST /api/v1/customer-app/auth/refresh

Access: Refresh credential | Success HTTP: 200

Fields and rules: refresh_token required. Rotation is atomic; expired, revoked or reused credentials cannot establish a session.

Request payload:

```
{
  "refresh_token": "<refresh_token>"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Rotate refresh credentials successful",
  "data": {
    "access_token": "<new_access_token>",
    "refresh_token": "<rotated_refresh_token>",
    "token_type": "Bearer",
    "expires_in_seconds": 900
  }
}
```

Representative failures: 401 REFRESH_TOKEN_INVALID; 401 SESSION_REVOKED. Common authentication/validation rules also apply where relevant.

API-06 | Validate current session

GET /api/v1/customer-app/auth/session

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Validate current session successful",
  "data": {
    "session_id": "session_001",
    "customer_id": "customer_001",
    "authenticated": true,
    "expires_at": "2026-10-08T10:15:00Z"
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-07 | Revoke current session

POST /api/v1/customer-app/auth/logout

Access: Customer | Success HTTP: 200

Fields and rules: Idempotent for a recognized session; do not accept a customer ID in the body.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Revoke current session successful",
  "data": {
    "logged_out": true
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

Conditional password option

Traceability: US-3 | UI-01

• These four operations are CONDITIONAL: include only if the client retains the password option shown on the login screen. OTP remains the core path.

• A verified password-setup/change proof is obtained through a purpose-bound OTP challenge extension. Do not reuse a normal access token as proof of recent verification.

• Use secure password hashing and an approved password policy. Reset requests must use generic responses; recovery delivery, proof issuance and expiry require a final design.

API-08 | Password login

POST /api/v1/customer-app/auth/password/login

Access: Customer pre-login | Success HTTP: 200

Fields and rules: identifier and password required; device_name optional. Conditional operation.

Request payload:

```
{
  "identifier": "customer@example.test",
  "password": "<password>",
  "device_name": "Customer Phone"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Password login successful",
  "data": {
    "customer": {
      "id": "customer_001"
    },
    "access_token": "<access_token>",
    "refresh_token": "<refresh_token>",
    "token_type": "Bearer",
    "expires_in_seconds": 900,
    "session_id": "session_001"
  }
}
```

Representative failures: 401 INVALID_CREDENTIALS; 429 RATE_LIMITED. Common authentication/validation rules also apply where relevant.

API-09 | Set or change password

PUT /api/v1/customer-app/auth/password

Access: Customer | Success HTTP: 200

Fields and rules: Both fields required. Recent verification proof is single-use and bound to password setup/change.

Request payload:

```
{
  "verification_token": "<purpose_bound_verification_proof>",
  "new_password": "<new_password>"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Set or change password successful",
  "data": {
    "password_updated": true
  }
}
```

Representative failures: 400 VERIFICATION_PROOF_INVALID; 422 PASSWORD_POLICY_FAILED. Common authentication/validation rules also apply where relevant.

API-10 | Request password recovery

POST /api/v1/customer-app/auth/password/reset/request

Access: Customer pre-login | Success HTTP: 202

Fields and rules: identifier required. Generic response regardless of account existence.

Request payload:

```
{
  "identifier": "customer@example.test"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Request password recovery successful",
  "data": {
    "request_received": true
  }
}
```

Representative failures: 422 VALIDATION_ERROR; 429 RATE_LIMITED. Common authentication/validation rules also apply where relevant.

API-11 | Complete password recovery

POST /api/v1/customer-app/auth/password/reset/confirm

Access: Recovery credential | Success HTTP: 200

Fields and rules: Both fields required. Proof must be purpose-bound, single-use and unexpired.

Request payload:

```
{
  "reset_token": "<reset_token>",
  "new_password": "<new_password>"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Complete password recovery successful",
  "data": {
    "password_reset": true,
    "existing_sessions_revoked": true
  }
}
```

Representative failures: 400 RESET_TOKEN_INVALID; 422 PASSWORD_POLICY_FAILED. Common authentication/validation rules also apply where relevant.

Customer profile

Traceability: US-6 | UI-08

• Only the authenticated customer's profile is accessible. Profile updates cannot set ownership, permissions, verified flags or account roles.

• Name must be nonblank. Address is optional; when provided validate city, postal code and country. Profile PATCH preserves omitted fields.

• Changing mobile/email requires verification of the new identifier and approved recent-authentication checks. Keep the old contact until verification completes; prevent duplicate verified contacts.

• Profile photos use a customer-owned READY attachment with PROFILE_PHOTO purpose. Deleting the profile photo detaches it subject to storage cleanup policy.

API-12 | View profile

GET /api/v1/customer-app/me

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "View profile successful",
  "data": {
    "id": "customer_001",
    "name": "Demo Customer",
    "mobile": "+9190000000001",
    "email": null,
    "mobile_verified": true,
    "email_verified": false,
    "photo": null,
    "address": {
      "line1": "Example Street",
      "city": "Hyderabad",
      "postal_code": "500001"
    }
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-13 | Update name/address

PATCH /api/v1/customer-app/me

Access: Customer | Success HTTP: 200

Fields and rules: At least one supported field required; both name and address are individually optional. Contacts must use the verified contact-change flow.

Request payload:

```
{
  "name": "Demo Customer",
  "address": {
    "line1": "Example Street",
    "city": "Hyderabad",
    "state": "Telangana",
    "postal_code": "500001",
    "country": "IN"
  }
}
```

Sample success response:

```
{
  "success": true,
  "message": "Update name/address successful",
  "data": {
    "id": "customer_001",
    "name": "Demo Customer",
    "address": {
      "line1": "Example Street",
      "city": "Hyderabad",
      "state": "Telangana",
      "postal_code": "500001",
      "country": "IN"
    }
  }
}
```

Representative failures: 422 VALIDATION_ERROR. Common authentication/validation rules also apply where relevant.

API-14 | Start contact change

POST /api/v1/customer-app/me/contact-change

Access: Customer | Success HTTP: 202

Fields and rules: Both fields required. channel is mobile or email. New identifier must not conflict with another account.

Request payload:

```
{
  "channel": "email",
  "new_identifier": "customer@example.test"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Start contact change successful",
  "data": {
    "challenge_id": "contact_challenge_001",
    "expires_in_seconds": 300
  }
}
```

Representative failures: 409 CONTACT_ALREADY_IN_USE; 429 RATE_LIMITED. Common authentication/validation rules also apply where relevant.

API-15 | Verify and apply new contact

POST /api/v1/customer-app/me/contact-change/verify

Access: Customer | Success HTTP: 200

Fields and rules: Both fields required. Apply the new contact atomically; recheck uniqueness.

Request payload:

```
{
  "challenge_id": "contact_challenge_001",
  "otp": "123456"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Verify and apply new contact successful",
  "data": {
    "email": "customer@example.test",
    "email_verified": true
  }
}
```

Representative failures: 400 OTP_INVALID; 400 OTP_EXPIRED; 409 CONTACT_ALREADY_IN_USE; 429 RATE_LIMITED. Common authentication/validation rules also apply where relevant.

API-16 | Set profile photo

PUT /api/v1/customer-app/me/photo

Access: Customer | Success HTTP: 200

Fields and rules: attachment_id required and must be READY, PROFILE_PHOTO and customer-owned.

Request payload:

```
{
  "attachment_id": "attachment_profile_001"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Set profile photo successful",
  "data": {
    "photo": {
      "attachment_id": "attachment_profile_001",
      "download_path": "/api/v1/customer-app/attachments/attachment_profile_001/download"
    }
  }
}
```

Representative failures: 422 ATTACHMENT_INVALID. Common authentication/validation rules also apply where relevant.

API-17 | Remove profile photo

DELETE /api/v1/customer-app/me/photo

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Remove profile photo successful",
  "data": {
    "removed": true
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

Vehicles and service history

Traceability: US-7, US-8, US-9 | UI-09

• Vehicle IDs must belong to the customer. Registration numbers are normalized and validated; duplicate active entries for the same customer are rejected.

• Required on create: registration_number, brand, model, fuel_type. Proposed optional fields: year, odometer_km, is_default. Fuel type must come from the approved enum; odometer is nonnegative and year is plausible.

• Exactly one default when vehicles exist is the proposed rule. Default changes are atomic; deletion of a default requires a replacement rule.

• Removal is blocked while active work exists. Use archival where necessary so historical jobs and invoices remain accessible.

API-18 | Add vehicle

POST /api/v1/customer-app/vehicles

Access: Customer | Success HTTP: 201

Request payload:

```
{
  "registration_number": "TS00AA0001",
  "brand": "Maruti Suzuki",
  "model": "Swift",
  "year": 2023,
  "fuel_type": "PETROL",
  "odometer_km": 25000,
  "is_default": true
}
```

Sample success response:

```
{
  "success": true,
  "message": "Add vehicle successful",
  "data": {
    "id": "vehicle_001",
    "registration_number": "TS00AA0001",
    "brand": "Maruti Suzuki",
    "model": "Swift",
    "year": 2023,
    "fuel_type": "PETROL",
    "odometer_km": 25000,
    "is_default": true
  }
}
```

Representative failures: 409 VEHICLE_ALREADY_EXISTS; 422 VALIDATION_ERROR. Common authentication/validation rules also apply where relevant.

API-19 | List vehicles

GET /api/v1/customer-app/vehicles

Access: Customer | Success HTTP: 200

Sample query: ?page=1&page_size=20

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "List vehicles successful",
  "data": {
    "items": [
      {
        "id": "vehicle_001",
        "registration_number": "TS00AA0001",
        "brand": "Maruti Suzuki",
        "model": "Swift",
        "year": 2023,
        "fuel_type": "PETROL",
        "odometer_km": 25000,
        "is_default": true
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```


Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-20 | View vehicle

GET /api/v1/customer-app/vehicles/{vehicle_id}

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:


```
{
  "success": true,
  "message": "View vehicle successful",
  "data": {
    "id": "vehicle_001",
    "registration_number": "TS00AA0001",
    "brand": "Maruti Suzuki",
    "model": "Swift",
    "year": 2023,
    "fuel_type": "PETROL",
    "odometer_km": 25000,
    "is_default": true
  }
}
```


Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-21 | Edit vehicle

PATCH /api/v1/customer-app/vehicles/{vehicle_id}

Access: Customer | Success HTTP: 200

Fields and rules: At least one editable field required. All create fields except is_default may be patched; use the dedicated default endpoint.

Request payload:


```
{
  "odometer_km": 26000,
  "year": 2023
}
```


Sample success response:

```
{
  "success": true,
  "message": "Edit vehicle successful",
  "data": {
    "id": "vehicle_001",
    "registration_number": "TS00AA0001",
    "brand": "Maruti Suzuki",
    "model": "Swift",
    "year": 2023,
    "fuel_type": "PETROL",
    "odometer_km": 26000,
    "is_default": true
  }
}
```

Representative failures: 409 VEHICLE_ALREADY_EXISTS; 422 VALIDATION_ERROR. Common authentication/validation rules also apply where relevant.

API-22 | Remove vehicle

DELETE /api/v1/customer-app/vehicles/{vehicle_id}

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Remove vehicle successful",
  "data": {
    "id": "vehicle_001",
    "removed": true
  }
}
```

Representative failures: 409 VEHICLE_HAS_ACTIVE_WORK. Common authentication/validation rules also apply where relevant.

API-23 | Choose default vehicle

PUT /api/v1/customer-app/vehicles/{vehicle_id}/default

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Choose default vehicle successful",
  "data": {
    "default_vehicle_id": "vehicle_001"
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-24 | View service history

GET /api/v1/customer-app/vehicles/{vehicle_id}/service-history

Access: Customer | Success HTTP: 200

Sample query: ?page=1&page_size=20

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "View service history successful",
  "data": {
    "items": [
      {
        "booking_id": "booking_001",
        "job_id": "job_001",
        "job_status": "COMPLETED",
        "completed_at": "2026-10-09T12:00:00+05:30",
        "garage": {
          "id": "garage_001",
          "name": "Example Garage"
        },
        "invoice_id": "invoice_001"
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```


Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

Secure attachments

Traceability: US-16, US-27, US-38 | UI-08, UI-10, UI-14, UI-20

• Supported purposes: PROFILE_PHOTO, BOOKING_PHOTO, SUPPORT_ATTACHMENT. Job photos are uploaded by garages, not customers.

• Photo defaults proposed for approval: JPEG/PNG/WebP, up to 5 MB each, maximum 5 booking photos. Support formats/limits require approval.

• Validate actual bytes, MIME type and size; sanitize filenames; scan as appropriate. Only READY, owned, purpose-compatible attachments can be linked.

• Downloads authorize access using ownership or the permitted linked job/context. Never expose permanent public URLs for private files. Deletion must respect links and retention policies.

API-25 | Upload attachment

POST /api/v1/customer-app/attachments

Access: Customer | Success HTTP: 201

Fields and rules: file and purpose required. Return READY only after required validation/scanning completes.

Request payload:

```
Multipart/form-data: file=<binary image>; purpose=BOOKING_PHOTO
```

Sample success response:

```
{
  "success": true,
  "message": "Upload attachment successful",
  "data": {
    "id": "attachment_001",
    "purpose": "BOOKING_PHOTO",
    "filename": "vehicle-front.jpg",
    "content_type": "image/jpeg",
    "size_bytes": 245000,
    "status": "READY"
  }
}
```

Representative failures: 413 ATTACHMENT_TOO_LARGE; 415 ATTACHMENT_TYPE_UNSUPPORTED; 422 ATTACHMENT_INVALID. Common authentication/validation rules also apply where relevant.

API-26 | Download authorized attachment

GET /api/v1/customer-app/attachments/{attachment_id}/download

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
Binary response: Content-Type: image/jpeg; Content-Disposition: inline; filename="vehicle-
  front.jpg"
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-27 | Remove unlinked attachment

DELETE /api/v1/customer-app/attachments/{attachment_id}

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Remove unlinked attachment successful",
  "data": {
    "id": "attachment_001",
    "removed": true
  }
}
```

Representative failures: 409 ATTACHMENT_IN_USE. Common authentication/validation rules also apply where relevant.

Availability and booking summary

Traceability: US-15, US-16, US-17, US-18, US-19 | UI-10

• Reject past appointment dates, inactive services and slots too short for selected service duration. Use each garage's IANA timezone; future means future at that garage.

• Availability is advisory. Quote creation does not reserve capacity in this proposal. Submission must atomically recheck capacity, service availability, vehicle ownership and quote validity.

• Selection can remain in the app until quote submission; no persistent selection API is required by the workbook.

• Estimated totals are computed server-side and are not final invoices. The quote binds customer, garage, vehicle, selected services, slot, notes and owned attachment IDs.

API-28 | Find appointment slots

GET /api/v1/customer-app/garages/{garage_id}/availability

Access: Customer | Success HTTP: 200

Sample query: ?date=2026-10-09&service_ids=service_001

Fields and rules: date and at least one service ID required.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Find appointment slots successful",
  "data": {
    "date": "2026-10-09",
    "timezone": "Asia/Kolkata",
    "slots": [
      {
        "id": "slot_001",
        "starts_at": "2026-10-09T10:00:00+05:30",
        "ends_at": "2026-10-09T12:00:00+05:30",
        "available": true
      }
    ]
  }
}
```

Representative failures: 422 INVALID_APPOINTMENT_DATE; 422 SERVICE_INACTIVE; 404 GARAGE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-29 | Validate selection and review summary

POST /api/v1/customer-app/bookings/quote

Access: Customer | Success HTTP: 200

Fields and rules: garage_id, vehicle_id, nonempty unique service_ids and slot_id required. notes and attachment_ids optional. Proposed notes limit: 2,000 characters.

Request payload:

```
{
  "garage_id": "garage_001",
  "vehicle_id": "vehicle_001",
  "service_ids": [
    "service_001"
  ],
  "slot_id": "slot_001",
  "notes": "Please inspect engine noise",
  "attachment_ids": [
    "attachment_001"
  ]
}
```

Sample success response:

```
{
  "success": true,
  "message": "Validate selection and review summary successful",
  "data": {
    "quote_id": "quote_001",
    "garage": {
      "id": "garage_001",
      "name": "Example Garage"
    },
    "vehicle": {
      "id": "vehicle_001",
      "registration_number": "TS00AA0001"
    },
    "services": [
      {
        "id": "service_001",
        "name": "Basic Service",
        "amount": 150000
      }
    ],
    "starts_at": "2026-10-09T10:00:00+05:30",
    "notes": "Please inspect engine noise",
    "attachment_ids": [
      "attachment_001"
    ],
    "estimated_total": 150000,
    "currency": "INR",
    "expires_at": "2026-10-08T15:10:00+05:30",
    "slot_reserved": false
  }
}
```


Representative failures: 409 SLOT_UNAVAILABLE; 422 SERVICE_INACTIVE; 422 ATTACHMENT_INVALID; 404 VEHICLE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

Booking management

Traceability: US-20, US-21, US-22, US-23, US-24 | UI-10, UI-11, UI-12

• Booking states: PENDING, ACCEPTED, REJECTED, CANCELLED, CONVERTED, EXPIRED. COMPLETED is a linked-job filter, not a booking state.

• Garages alone accept/reject bookings and convert them into jobs. Customers cannot supply garage decision timestamps or change status directly.

• Proposed baseline: customer cancellation is allowed only while PENDING, before work begins. Accepted-booking cancellation, cutoff periods and fees need client approval. Expose can_cancel and a reason when unavailable.

• Idempotency keys are scoped to customer and operation. Same key and same payload return the original result; same key with a different payload returns 409. Capacity and booking creation occur atomically.

API-30 | Submit booking request

POST /api/v1/customer-app/bookings

Access: Customer | Success HTTP: 201

Required extra header: Idempotency-Key: <unique_key>

Fields and rules: quote_id required. customer_id is derived from the session and is not accepted from the client.

Request payload:

```
{
  "quote_id": "quote_001"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Submit booking request successful",
  "data": {
    "id": "booking_001",
    "reference": "GFC-B-000001",
    "garage_id": "garage_001",
    "vehicle_id": "vehicle_001",
    "status": "PENDING",
    "starts_at": "2026-10-09T10:00:00+05:30",
    "estimated_total": 150000,
    "currency": "INR",
    "job_id": null
  }
}
```

Representative failures: 409 SLOT_UNAVAILABLE; 409 QUOTE_EXPIRED; 409 IDEMPOTENCY_CONFLICT. Common authentication/validation rules also apply where relevant.

API-31 | List own bookings

GET /api/v1/customer-app/bookings

Access: Customer | Success HTTP: 200

Sample query: ?status=PENDING&page=1&page_size=20

Fields and rules: Optional status, vehicle_id, garage_id and linked_job_status filters. Completed tab: linked_job_status=COMPLETED.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "List own bookings successful",
  "data": {
    "items": [
      {
        "id": "booking_001",
        "reference": "GFC-B-000001",
        "status": "PENDING",
        "garage": {
          "id": "garage_001",
          "name": "Example Garage"
        },
        "starts_at": "2026-10-09T10:00:00+05:30",
        "job_id": null
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```


Representative failures: 422 INVALID_STATUS_FILTER. Common authentication/validation rules also apply where relevant.

API-32 | View booking and garage decision

GET /api/v1/customer-app/bookings/{booking_id}

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:


```
{
  "success": true,
  "message": "View booking and garage decision successful",
  "data": {
    "id": "booking_001",
    "reference": "GFC-B-000001",
    "status": "ACCEPTED",
    "vehicle_id": "vehicle_001",
    "garage_id": "garage_001",
    "service_ids": [
      "service_001"
    ],
    "notes": "Please inspect engine noise",
    "attachment_ids": [
      "attachment_001"
    ],
    "rejection_reason": null,
    "accepted_at": "2026-10-08T16:00:00+05:30",
    "job_id": null,
    "can_cancel": false,
    "cancellation_unavailable_reason": "Only pending bookings can be cancelled"
  }
}
```


Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-33 | Read booking lifecycle

GET /api/v1/customer-app/bookings/{booking_id}/timeline

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Read booking lifecycle successful",
  "data": {
    "items": [
      {
        "status": "PENDING",
        "timestamp": "2026-10-08T15:00:00+05:30",
        "note": "Booking submitted"
      },
      {
        "status": "ACCEPTED",
        "timestamp": "2026-10-08T16:00:00+05:30",
        "note": "Appointment confirmed"
      }
    ]
  }
}
```


Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-34 | Cancel eligible booking

POST /api/v1/customer-app/bookings/{booking_id}/cancel

Access: Customer | Success HTTP: 200

Fields and rules: reason required, nonblank, proposed maximum 500 characters. Sample cancellation uses a separate PENDING booking_002; booking_001 was accepted.

Request payload:


```
{
  "reason": "Change of plans"
}
```


Sample success response:


```
{
  "success": true,
  "message": "Cancel eligible booking successful",
  "data": {
    "id": "booking_002",
    "status": "CANCELLED",
    "cancellation_reason": "Change of plans",
    "cancelled_at": "2026-10-08T15:30:00+05:30"
  }
}
```


Representative failures: 409 CANCELLATION_NOT_ALLOWED. Common authentication/validation rules also apply where relevant.

Published job tracking - read only

Traceability: US-24, US-25, US-26, US-27 | UI-13, UI-14

• Job states: OPEN, IN_PROGRESS, WAITING_FOR_PARTS, QUALITY_CHECK, READY, COMPLETED, CANCELLED.

• Customer access is read-only. No customer job-create, job-update, status-change, labour/parts mutation or garage-photo upload operation is permitted.

• Only published/customer-visible findings, notes, photos, services, parts and labour are returned. Exclude internal notes, cost margins, staff-only fields and unrelated customer information.

• Timeline events are chronological. A converted booking exposes its linked garage-created job and job creation timestamp.

API-35 | List accessible jobs

GET /api/v1/customer-app/jobs

Access: Customer | Success HTTP: 200

Sample query: ?status=IN_PROGRESS&page=1&page_size=20

Fields and rules: Optional status and vehicle_id filters.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "List accessible jobs successful",
  "data": {
    "items": [
      {
        "id": "job_001",
        "job_number": "GFC-J-000001",
        "booking_id": "booking_001",
        "vehicle_id": "vehicle_001",
        "status": "IN_PROGRESS"
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```

Representative failures: 422 INVALID_STATUS_FILTER. Common authentication/validation rules also apply where relevant.

API-36 | Read published job details

GET /api/v1/customer-app/jobs/{job_id}

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Read published job details successful",
  "data": {
    "id": "job_001",
    "job_number": "GFC-J-000001",
    "booking_id": "booking_001",
    "garage_id": "garage_001",
    "vehicle_id": "vehicle_001",
    "status": "IN_PROGRESS",
    "created_at": "2026-10-09T10:00:00+05:30",
    "inspection_findings": [
      {
        "description": "Engine inspection completed"
      }
    ],
    "services": [
      {
        "name": "Basic Service"
      }
    ],
    "parts": [
      {
        "name": "Engine oil",
        "quantity": 1
      }
    ],
    "labour": [
      {
        "description": "Oil change"
      }
    ],
    "photos": [
      {
        "attachment_id": "job_photo_001"
      }
    ],
    "invoice_id": null
  }
}
```


Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-37 | Read status timeline

GET /api/v1/customer-app/jobs/{job_id}/timeline

Access: Customer | Success HTTP: 200

Sample query: ?page=1&page_size=20

Request payload:

```
No request body.
```

Sample success response:


```
{
  "success": true,
  "message": "Read status timeline successful",
  "data": {
    "items": [
      {
        "status": "OPEN",
        "timestamp": "2026-10-09T10:00:00+05:30",
        "note": "Vehicle received",
        "attachment_ids": []
      },
      {
        "status": "IN_PROGRESS",
        "timestamp": "2026-10-09T10:15:00+05:30",
        "note": "Inspection started",
        "attachment_ids": [
          "job_photo_001"
        ]
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 2
  }
}
```


Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

Invoices and secure PDF downloads

Traceability: US-28, US-29, US-30 | UI-15

• Expose only the customer's published final invoice after the job meets completion/publication rules. Missing historical bills are not fabricated by this specification.

• Payment display states: UNPAID, PENDING, PAID, FAILED, REFUNDED. These must be derived from server-confirmed financial records and reconciled attempts.

• Amounts are integer paise in INR. Example: subtotal 150000 + tax 27000 - discount 0 = total 177000. Tax/discount ordering and legal invoice fields must follow the approved billing policy.

• Invoice PDF is an authenticated binary response. The mobile app uses its native share action after download; no separate public share endpoint is required.

API-38 | List final invoices

GET /api/v1/customer-app/invoices

Access: Customer | Success HTTP: 200

Sample query: ?payment_status=UNPAID&page=1&page_size=20

Fields and rules: Optional payment_status, job_id and vehicle_id filters.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "List final invoices successful",
  "data": {
    "items": [
      {
        "id": "invoice_001",
        "invoice_number": "GFC-I-000001",
        "job_id": "job_001",
        "total_amount": 177000,
        "currency": "INR",
        "payment_status": "UNPAID"
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-39 | View invoice charges

GET /api/v1/customer-app/invoices/{invoice_id}

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "View invoice charges successful",
  "data": {
    "id": "invoice_001",
    "invoice_number": "GFC-I-000001",
    "job_id": "job_001",
    "total_amount": 177000,
    "currency": "INR",
    "payment_status": "UNPAID",
    "garage_id": "garage_001",
    "vehicle_id": "vehicle_001",
    "line_items": [
      {
        "type": "SERVICE",
        "description": "Inspection",
        "quantity": 1,
        "unit_amount": 50000,
        "line_total": 50000
      },
      {
        "type": "PART",
        "description": "Engine oil",
        "quantity": 1,
        "unit_amount": 80000,
        "line_total": 80000
      },
      {
        "type": "LABOUR",
        "description": "Oil change labour",
        "quantity": 1,
        "unit_amount": 20000,
        "line_total": 20000
      }
    ],
    "subtotal_amount": 150000,
    "discount_amount": 0,
    "taxes": [
      {
        "name": "GST",
        "rate_percent": 18,
        "amount": 27000
      }
    ],
    "paid_amount": 0,
    "balance_due": 177000
  }
}
```


Representative failures: 404 INVOICE_NOT_AVAILABLE. Common authentication/validation rules also apply where relevant.

API-40 | Download/share invoice PDF

GET /api/v1/customer-app/invoices/{invoice_id}/pdf

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
Binary response: Content-Type: application/pdf; Content-Disposition: attachment;
  filename="GFC-I-000001.pdf"
```

Representative failures: 404 INVOICE_NOT_AVAILABLE. Common authentication/validation rules also apply where relevant.

Payments and safe retries

Traceability: US-31, US-32 | UI-16

• Payment provider is undecided. Checkout fields are illustrative placeholders until the selected provider's official contract is adopted.

• The backend calculates the invoice balance, currency and gateway order. Never accept client-supplied settled status or amount.

• Retry uses the same creation endpoint with a new idempotency key after a confirmed failed attempt. Reuse an active valid attempt; reject creation for a fully settled invoice.

• Only a verified provider webhook or server-side gateway verification may confirm settlement. Verify order mapping, amount and currency; deduplicate events; make invoice/payment changes transactional.

• return_path is optional and restricted to approved in-app routes; never accept an arbitrary external redirect. Partial payments/refunds are outside the approved scope until separately decided.

API-41 | Start or retry payment

POST /api/v1/customer-app/invoices/{invoice_id}/payment-attempts

Access: Customer | Success HTTP: 201

Required extra header: Idempotency-Key: <unique_key>

Fields and rules: Body may be {}. Return 200 if reusing an existing active attempt rather than creating one.

Request payload:

```
{
  "return_path": "/payments/result"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Start or retry payment successful",
  "data": {
    "id": "payment_001",
    "invoice_id": "invoice_001",
    "amount": 177000,
    "currency": "INR",
    "status": "PENDING",
    "checkout": {
      "provider": "<configured_provider>",
      "order_id": "<provider_order_id>",
      "session_token": "<short_lived_checkout_token>"
    }
  }
}
```

Representative failures: 409 INVOICE_ALREADY_PAID; 409 IDEMPOTENCY_CONFLICT; 502 PAYMENT_PROVIDER_UNAVAILABLE. Common authentication/validation rules also apply where relevant.

API-42 | Check confirmed payment result

GET /api/v1/customer-app/payments/{payment_id}

Access: Customer | Success HTTP: 200

Fields and rules: Example shows the successful retry payment_002; payment_001 appears as failed in attempt history.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Check confirmed payment result successful",
  "data": {
    "id": "payment_002",
    "invoice_id": "invoice_001",
    "amount": 177000,
    "currency": "INR",
    "status": "PAID",
    "paid_at": "2026-10-09T12:30:00+05:30",
    "receipt_available": true
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-43 | Read payment attempt history

GET /api/v1/customer-app/invoices/{invoice_id}/payment-attempts

Access: Customer | Success HTTP: 200

Sample query: ?page=1&page_size=20

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Read payment attempt history successful",
  "data": {
    "items": [
      {
        "id": "payment_001",
        "status": "FAILED",
        "failure_code": "PAYMENT_DECLINED",
        "amount": 177000,
        "currency": "INR"
      },
      {
        "id": "payment_002",
        "status": "PAID",
        "amount": 177000,
        "currency": "INR"
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 2
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-44 | Download settled payment receipt

GET /api/v1/customer-app/payments/{payment_id}/receipt

Access: Customer | Success HTTP: 200

Fields and rules: Available only for a server-confirmed paid attempt.

Request payload:

```
No request body.
```

Sample success response:

```
Binary PDF response: Content-Type: application/pdf; Content-Disposition: attachment;
  filename="payment-receipt.pdf"
```

Representative failures: 409 RECEIPT_NOT_AVAILABLE. Common authentication/validation rules also apply where relevant.

Notifications and unified activity center

Traceability: US-23, US-33, US-34 | UI-04, UI-17

• Categories: BOOKINGS, JOBS, INVOICES, MESSAGES, PAYMENTS, REVIEWS, SUPPORT. Each event includes timestamp, read state and authorized linked-record context.

• Create in-app activity independently of push delivery. Booking accepted/rejected, job created/status changed, invoice published, payment outcome, message/review/support events feed this center.

• Device tokens belong to an app installation bound to the current customer/session. Logout detaches it as appropriate; do not keep sending a previous customer's private events on a shared phone.

API-45 | Register/update push installation

PUT /api/v1/customer-app/devices/{installation_id}

Access: Customer | Success HTTP: 200

Fields and rules: platform and push_token required; platform enum ANDROID or IOS.

Request payload:

```
{
  "platform": "ANDROID",
  "push_token": "<device_push_token>"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Register/update push installation successful",
  "data": {
    "installation_id": "installation_001",
    "registered": true
  }
}
```

Representative failures: 422 DEVICE_INVALID. Common authentication/validation rules also apply where relevant.

API-46 | Unregister push installation

DELETE /api/v1/customer-app/devices/{installation_id}

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Unregister push installation successful",
  "data": {
    "unregistered": true
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-47 | List/filter activity

GET /api/v1/customer-app/notifications

Access: Customer | Success HTTP: 200

Sample query: ?category=BOOKINGS&unread_only=true&page=1&page_size=20

Fields and rules: Omit category for All. unread_count is the count matching the selected category, independent of pagination.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "List/filter activity successful",
  "data": {
    "items": [
      {
        "id": "notification_001",
        "category": "BOOKINGS",
        "event_type": "BOOKING_ACCEPTED",
        "title": "Booking accepted",
        "body": "Example Garage accepted your booking",
        "related": {
          "type": "BOOKING",
          "id": "booking_001"
        },
        "is_read": false,
        "created_at": "2026-10-08T16:00:00+05:30"
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1,
    "unread_count": 1
  }
}
```


Representative failures: 422 INVALID_CATEGORY. Common authentication/validation rules also apply where relevant.

API-48 | Mark notification read

PATCH /api/v1/customer-app/notifications/{notification_id}

Access: Customer | Success HTTP: 200

Fields and rules: is_read required and must be true for this contract.

Request payload:


```
{
  "is_read": true
}
```


Sample success response:


```
{
  "success": true,
  "message": "Mark notification read successful",
  "data": {
    "id": "notification_001",
    "is_read": true,
    "read_at": "2026-10-08T16:05:00+05:30"
  }
}
```


Representative failures: 422 VALIDATION_ERROR. Common authentication/validation rules also apply where relevant.

API-49 | Mark all/filtered activity read

POST /api/v1/customer-app/notifications/read-all

Access: Customer | Success HTTP: 200

Fields and rules: category optional; {} marks all categories. Operate on current customer's records only.

Request payload:


```
{
  "category": "BOOKINGS"
}
```


Sample success response:


```
{
  "success": true,
  "message": "Mark all/filtered activity read successful",
  "data": {
    "updated_count": 5
  }
}
```


Representative failures: 422 INVALID_CATEGORY. Common authentication/validation rules also apply where relevant.

Garage conversations

Traceability: UI-only requirement | UI-18

• This is UI-required scope even though the workbook has no separate numbered messaging story.

• Start conversations using exactly one customer-owned booking_id or job_id. Derive the garage from that record. Reuse an existing conversation for the context; define continuity when a booking converts to a job.

• Only context participants can read/send. Proposed messages are plain text with maximum 2,000 characters. Post-completion/cancellation sending permissions need approval.

• client_message_id is required for sends and deduplicated per sender/conversation. Cursor pagination is chronological and stable. Matching garage-side participant APIs are required.

API-50 | Open or reuse contextual conversation

POST /api/v1/customer-app/conversations

Access: Customer | Success HTTP: 201

Fields and rules: Exactly one booking_id or job_id required. Return 200 when reusing an existing conversation.

Request payload:

```
{
  "booking_id": "booking_001"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Open or reuse contextual conversation successful",
  "data": {
    "id": "conversation_001",
    "garage_id": "garage_001",
    "booking_id": "booking_001",
    "job_id": null,
    "can_send": true
  }
}
```

Representative failures: 422 CONVERSATION_CONTEXT_INVALID; 403 MESSAGING_NOT_ALLOWED. Common authentication/validation rules also apply where relevant.

API-51 | List conversations

GET /api/v1/customer-app/conversations

Access: Customer | Success HTTP: 200

Sample query: ?page=1&page_size=20

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "List conversations successful",
  "data": {
    "items": [
      {
        "id": "conversation_001",
        "garage_name": "Example Garage",
        "booking_id": "booking_001",
        "unread_count": 1,
        "last_message": "Your appointment is confirmed"
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-52 | Read message history

GET /api/v1/customer-app/conversations/{conversation_id}/messages

Access: Customer | Success HTTP: 200

Sample query: ?limit=30

Fields and rules: cursor optional; limit proposed range 1..100.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Read message history successful",
  "data": {
    "items": [
      {
        "id": "message_001",
        "sender_role": "GARAGE",
        "text": "Your appointment is confirmed",
        "created_at": "2026-10-08T16:00:00+05:30"
      }
    ],
    "next_cursor": null
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-53 | Send customer message

POST /api/v1/customer-app/conversations/{conversation_id}/messages

Access: Customer | Success HTTP: 201

Fields and rules: Both fields required. sender_role and timestamp are server-controlled.

Request payload:

```
{
  "client_message_id": "client_message_001",
  "text": "Thank you. I will arrive on time."
}
```

Sample success response:

```
{
  "success": true,
  "message": "Send customer message successful",
  "data": {
    "id": "message_002",
    "sender_role": "CUSTOMER",
    "text": "Thank you. I will arrive on time.",
    "created_at": "2026-10-08T16:10:00+05:30"
  }
}
```

Representative failures: 409 CONVERSATION_CLOSED; 409 MESSAGE_ID_CONFLICT; 422 MESSAGE_INVALID. Common authentication/validation rules also apply where relevant.

API-54 | Advance read marker

POST /api/v1/customer-app/conversations/{conversation_id}/read

Access: Customer | Success HTTP: 200

Fields and rules: last_read_message_id required and must belong to this conversation; markers cannot move backwards.

Request payload:

```
{
  "last_read_message_id": "message_001"
}
```

Sample success response:

```
{
  "success": true,
  "message": "Advance read marker successful",
  "data": {
    "last_read_message_id": "message_001",
    "unread_count": 0
  }
}
```

Representative failures: 422 MESSAGE_INVALID. Common authentication/validation rules also apply where relevant.

Ratings and reviews

Traceability: US-35, US-36 | UI-19

• Only the customer owning a COMPLETED job may review it. Enforce one review per completed job; derive garage_id and customer_id server-side.

• rating is required integer 1..5; comment optional, proposed maximum 2,000 characters. Author may edit subject to an approved edit window.

• Examples propose PENDING moderation before publication and after editing. Public endpoints exclude unapproved content. Define rating aggregation and moderation policy before implementation.

API-55 | Submit rating/review

POST /api/v1/customer-app/jobs/{job_id}/review

Access: Customer | Success HTTP: 201

Request payload:

```
{
  "rating": 5,
  "comment": "Good service and clear communication."
}
```

Sample success response:

```
{
  "success": true,
  "message": "Submit rating/review successful",
  "data": {
    "id": "review_001",
    "job_id": "job_001",
    "garage_id": "garage_001",
    "rating": 5,
    "comment": "Good service and clear communication.",
    "moderation_status": "PENDING",
    "published": false
  }
}
```

Representative failures: 409 JOB_NOT_COMPLETED; 409 REVIEW_ALREADY_EXISTS; 422 REVIEW_INVALID. Common authentication/validation rules also apply where relevant.

API-56 | View own reviews/edit eligibility

GET /api/v1/customer-app/me/reviews

Access: Customer | Success HTTP: 200

Sample query: ?page=1&page_size=20

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "View own reviews/edit eligibility successful",
  "data": {
    "items": [
      {
        "id": "review_001",
        "job_id": "job_001",
        "garage_id": "garage_001",
        "rating": 5,
        "comment": "Good service and clear communication.",
        "moderation_status": "PENDING",
        "published": false,
        "can_edit": true
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-57 | Edit own review

PATCH /api/v1/customer-app/reviews/{review_id}

Access: Customer | Success HTTP: 200

Fields and rules: At least one of rating or comment required. Only author can edit.

Request payload:

```
{
  "rating": 4,
  "comment": "Good service; delivery took longer than expected."
}
```

Sample success response:

```
{
  "success": true,
  "message": "Edit own review successful",
  "data": {
    "id": "review_001",
    "rating": 4,
    "comment": "Good service; delivery took longer than expected.",
    "moderation_status": "PENDING",
    "published": false
  }
}
```

Representative failures: 409 REVIEW_EDIT_WINDOW_CLOSED; 422 REVIEW_INVALID. Common authentication/validation rules also apply where relevant.

Customer support tickets and replies

Traceability: US-38 | UI-20

• Ticket requires category, subject and description. Proposed text limits: subject 200 characters; description 5,000; reply 2,000. Categories and attachment limits require approval.

• Optional linked booking/job/invoice/payment IDs must be owned by the customer and mutually consistent; general issues need no linked IDs.

• Customers view status and external replies only; they cannot set internal ticket status or access internal support notes. Reopening/closing policy requires approval.

API-58 | Create support request

POST /api/v1/customer-app/support/tickets

Access: Customer | Success HTTP: 201

Request payload:

```
{
  "category": "PAYMENT",
  "subject": "Payment status not updated",
  "description": "Payment completed but the invoice still shows pending.",
  "booking_id": "booking_001",
  "job_id": "job_001",
  "invoice_id": "invoice_001",
  "payment_id": "payment_002",
  "attachment_ids": []
}
```

Sample success response:

```
{
  "success": true,
  "message": "Create support request successful",
  "data": {
    "id": "ticket_001",
    "reference": "GFC-S-000001",
    "status": "OPEN",
    "created_at": "2026-10-09T12:40:00+05:30"
  }
}
```

Representative failures: 422 LINKED_RECORD_INVALID; 422 ATTACHMENT_INVALID; 422 VALIDATION_ERROR. Common authentication/validation rules also apply where relevant.

API-59 | List own tickets

GET /api/v1/customer-app/support/tickets

Access: Customer | Success HTTP: 200

Sample query: ?status=OPEN&page=1&page_size=20

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "List own tickets successful",
  "data": {
    "items": [
      {
        "id": "ticket_001",
        "reference": "GFC-S-000001",
        "subject": "Payment status not updated",
        "status": "OPEN"
      }
    ],
    "page": 1,
    "page_size": 20,
    "total": 1
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-60 | View ticket status/context

GET /api/v1/customer-app/support/tickets/{ticket_id}

Access: Customer | Success HTTP: 200

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "View ticket status/context successful",
  "data": {
    "id": "ticket_001",
    "category": "PAYMENT",
    "subject": "Payment status not updated",
    "description": "Payment completed but the invoice still shows pending.",
    "status": "IN_PROGRESS",
    "booking_id": "booking_001",
    "job_id": "job_001",
    "invoice_id": "invoice_001",
    "payment_id": "payment_002",
    "attachment_ids": []
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-61 | Read customer-visible support replies

GET /api/v1/customer-app/support/tickets/{ticket_id}/messages

Access: Customer | Success HTTP: 200

Sample query: ?limit=30

Fields and rules: cursor optional; limit proposed range 1..100.

Request payload:

```
No request body.
```

Sample success response:

```
{
  "success": true,
  "message": "Read customer-visible support replies successful",
  "data": {
    "items": [
      {
        "id": "support_message_001",
        "sender_role": "SUPPORT",
        "text": "We are checking your payment.",
        "created_at": "2026-10-09T13:00:00+05:30"
      }
    ],
    "next_cursor": null
  }
}
```

Representative failures: 401 AUTH_REQUIRED; 404 RESOURCE_NOT_FOUND. Common authentication/validation rules also apply where relevant.

API-62 | Reply to support

POST /api/v1/customer-app/support/tickets/{ticket_id}/messages

Access: Customer | Success HTTP: 201

Fields and rules: client_message_id and nonblank text required; attachment_ids optional and customer-owned.

Request payload:

```
{
  "client_message_id": "support_client_message_001",
  "text": "Please share an update.",
  "attachment_ids": []
}
```

Sample success response:

```
{
  "success": true,
  "message": "Reply to support successful",
  "data": {
    "id": "support_message_002",
    "sender_role": "CUSTOMER",
    "text": "Please share an update.",
    "created_at": "2026-10-09T13:10:00+05:30"
  }
}
```


Representative failures: 409 TICKET_REPLY_NOT_ALLOWED; 409 MESSAGE_ID_CONFLICT; 422 ATTACHMENT_INVALID. Common authentication/validation rules also apply where relevant.

4. Customer-only scope boundary

Included: customer registration/login/session flows, personal profile/contact verification/photos, owned vehicles/history, authenticated booking availability and quote submission, owned bookings, read-only published jobs, owned invoices/payments, devices/activity, contextual messages, own reviews and support tickets.

Excluded: public garage search/profile/service catalogue/reviews, popular service discovery, public FAQs/help/privacy/terms URLs, garage-side decision or job mutation URLs, administrative URLs and payment-provider callback URLs.

The authenticated garage availability operation remains because it is used to schedule the customer's booking. It is not a public garage listing or owner operation.

Customer authentication is the only pre-login exception. Requiring a customer access token for registration or initial login would prevent those flows from working. No general public browsing endpoint is included.

External dependencies may still exist without appearing in this customer-only catalogue: clients must obtain approved service/garage selections elsewhere, and settlement must still be verified server-side. This scope restriction does not grant customers any garage/admin privileges.

This is a proposed requirements baseline, not an implementation audit. Existing garage-owned customer records must not be mistaken for verified customer login identities.

5. Story and screen traceability

Source stories: included and intentionally excluded

Story	Source epic	Priority	Covering API IDs
US-1	Authentication	Must Have	API-01, API-02, API-03, API-04, API-05, API-06, API-07
US-2	Authentication	Must Have	API-01, API-02, API-03, API-04, API-05, API-06, API-07
US-3	Authentication	Must Have	API-01, API-02, API-03, API-04, API-05, API-06, API-07, API-08, API-09, API-10, API-11
US-4	Authentication	Must Have	API-01, API-02, API-03, API-04, API-05, API-06, API-07
US-5	Authentication	Must Have	API-01, API-02, API-03, API-04, API-05, API-06, API-07
US-6	Customer Profile	Must Have	API-12, API-13, API-14, API-15, API-16, API-17
US-7	Vehicle Management	Must Have	API-18, API-19, API-20, API-21, API-22, API-23, API-24
US-8	Vehicle Management	Should Have	API-18, API-19, API-20, API-21, API-22, API-23, API-24
US-9	Vehicle Management	Should Have	API-18, API-19, API-20, API-21, API-22, API-23, API-24
US-10	Garage Search	Must Have	Excluded: public discovery/content scope
US-11	Garage Search	Should Have	Excluded: public discovery/content scope
US-12	Garage Details	Must Have	Excluded: public discovery/content scope
US-13	Garage Details	Must Have	Excluded: public discovery/content scope
US-14	Garage Details	Must Have	Excluded: public discovery/content scope
US-15	Service Selection	Must Have	API-28, API-29
US-16	Service Selection	Should Have	API-25, API-26, API-27, API-28, API-29
US-17	Scheduling	Must Have	API-28, API-29
US-18	Scheduling	Must Have	API-28, API-29
US-19	Booking Management	Must Have	API-28, API-29
US-20	Booking Management	Must Have	API-30, API-31, API-32, API-33, API-34
US-21	Booking Management	Must Have	API-30, API-31, API-32, API-33, API-34
US-22	Booking Management	Should Have	API-30, API-31, API-32, API-33, API-34
US-23	Garage Decision	Must Have	API-30, API-31, API-32, API-33, API-34, API-45, API-46, API-47, API-48, API-49
US-24	Job Card Tracking	Must Have	API-30, API-31, API-32, API-33, API-34, API-35, API-36, API-37
US-25	Job Card Tracking	Must Have	API-35, API-36, API-37
US-26	Job Card Tracking	Must Have	API-35, API-36, API-37
US-27	Job Card Tracking	Should Have	API-25, API-26, API-27, API-35, API-36, API-37
US-28	Invoice Management	Must Have	API-38, API-39, API-40
US-29	Invoice Management	Must Have	API-38, API-39, API-40
US-30	Invoice Management	Should Have	API-38, API-39, API-40
US-31	Payments	Must Have	API-41, API-42, API-43, API-44
US-32	Payments	Should Have	API-41, API-42, API-43, API-44
US-33	Notifications	Must Have	API-45, API-46, API-47, API-48, API-49
US-34	Notifications	Should Have	API-45, API-46, API-47, API-48, API-49
US-35	Ratings and Reviews	Should Have	API-55, API-56, API-57
US-36	Ratings and Reviews	Should Have	API-55, API-56, API-57
US-37	Support and Static Content	Should Have	Excluded: public discovery/content scope
US-38	Support and Static Content	Should Have	API-25, API-26, API-27, API-58, API-59, API-60, API-61, API-62

US-10 through US-14 and US-37 are intentionally excluded as public discovery/content requirements. US-15 covers selected services in the booking quote, not the excluded public catalogue. US-23 retains customer-visible notifications; garage decision URLs are excluded.

Source screens: customer-only coverage

Screen	Source screen	Covering API IDs
UI-01	Login	API-01, API-02, API-03, API-04, API-05, API-06, API-07, API-08, API-09, API-10, API-11
UI-02	OTP Verification	API-01, API-02, API-03, API-04, API-05, API-06, API-07
UI-03	Registration	API-01, API-02, API-03, API-04, API-05, API-06, API-07
UI-04	Customer Home	API-45, API-46, API-47, API-48, API-49
UI-05	Search Garages	Excluded: public discovery scope
UI-06	Garage Profile	Excluded: public discovery scope
UI-07	Service Details	Excluded: public discovery scope
UI-08	Personal Details	API-12, API-13, API-14, API-15, API-16, API-17, API-25, API-26, API-27
UI-09	My Vehicles	API-18, API-19, API-20, API-21, API-22, API-23, API-24
UI-10	Booking Request	API-25, API-26, API-27, API-28, API-29, API-30, API-31, API-32, API-33, API-34
UI-11	Confirmation	API-30, API-31, API-32, API-33, API-34
UI-12	My Bookings	API-30, API-31, API-32, API-33, API-34
UI-13	Job Status	API-35, API-36, API-37
UI-14	Job Timeline	API-25, API-26, API-27, API-35, API-36, API-37
UI-15	Invoice Details	API-38, API-39, API-40
UI-16	Payment Result	API-41, API-42, API-43, API-44
UI-17	Unified Activity Center	API-45, API-46, API-47, API-48, API-49
UI-18	Garage Communication	API-50, API-51, API-52, API-53, API-54
UI-19	Rating and Review	API-55, API-56, API-57
UI-20	Help and Support	API-25, API-26, API-27, API-58, API-59, API-60, API-61, API-62
UI-21	Logout	API-01, API-02, API-03, API-04, API-05, API-06, API-07

Explicit UI additions covered: profile photo; default vehicle; garage communication; review editing; activity filters. Booking Completed is derived from linked job completion.

UI-05, UI-06 and UI-07 public discovery/catalogue operations are excluded. UI-04 covers notification/customer shortcuts only; public home discovery is excluded. UI-20 covers owned support tickets/replies only, not public content.

6. Approval decision register

ID	Decision	Client confirmation required
D-01	Registration contact	Approve mobile OR email registration, or revise stories to mobile-required registration.
D-02	Password option	Include conditional password login/setup/recovery APIs or remove the password UI option.
D-03	OTP and sessions	Approve expiry, resend/attempt limits, refresh lifetime, recent-verification rules and contact-change protections.
D-04	Scheduling	Approve capacity model, quote expiry, slot reservation behavior and timezone rules. Proposed quote does not reserve a slot.
D-05	Cancellation	Approve pending-only cancellation or define accepted-booking cutoff/fees/refunds.
D-06	Attachments	Approve photo types, 5 MB/photo, five booking photos, support formats, scanning and retention.
D-07	Payments	Choose provider; approve retry/reconciliation, receipts, full/partial payment scope and refund display rules.
D-08	Reviews	Approve moderation-before-publication, re-moderation after edits, edit window and public reviewer display policy.
D-09	Messaging	Approve eligible contexts, post-completion/cancellation permissions, continuity on job conversion and retention.
D-10	Customer support	Approve ticket categories/statuses and reply/closure policy. Public content endpoints are outside this revision.
D-11	Vehicle ownership	Approve linking verified customer identities to pre-existing garage vehicle/customer records; define duplicate and dispute handling.
D-12	Billing publication	Approve final invoice visibility, taxes/discounts and legal invoice fields. No historical bill repair is authorized.

All proposed defaults remain unapproved until confirmed. Open decisions may change request fields, endpoint behavior or delivery scope.

7. Client sign-off

Approval requested: customer-only API scope, explicitly excluded public requirements, payload/response direction, role boundaries and the decisions above.

Approval is for this requirements baseline only. Implementation effort, delivery schedule, provider costs and production changes require separate agreement.

Client / organization: _____

Reviewer name / role: _____

Decision: Approve / Approve with changes / Revise

Approved decisions / requested changes: _____

_____

Signature / written approval reference: _____

Approval date: _____